

Student

I keep seeing views referenced in data warehouses. Someone told me a view is just a saved query. But then I also hear about materialized views which are different. Can you explain both?

ChatGPT • AI Assistant

CONCEPT CARD

Both are named, reusable SELECT queries stored in the database — but they work very differently under the hood.

Regular View: a saved SELECT query with a name. It is a **virtual table** — it stores NO data. Every time you query a view, the database re-executes the underlying SELECT in real time. Think of it as a macro: the view definition is substituted into your query at execution time.

Materialized View: a saved SELECT query whose result is **physically stored on disk** as a real table. Reads are instant because no query executes — you are reading pre-computed rows. But the data goes stale: the stored result does not automatically reflect changes to the source tables until you refresh it.

🗨️ ANALOGY

A regular view is like a recipe card — every time you want the dish, you cook it fresh from scratch using the current ingredients. A materialized view is like a meal you cooked in advance and stored in the fridge — it is ready instantly, but it reflects what the ingredients were when you cooked it, not what is in the fridge today.

🌐 REAL WORLD APPLICATION

In data engineering, views are the standard tool for: encapsulating join logic, hiding sensitive columns, and creating a stable interface for downstream consumers. Materialized views are the standard tool for: pre-computing expensive aggregations that power dashboards, avoiding repeated heavy JOIN scans on fact tables, and caching summary results that are acceptable to be a few hours stale.

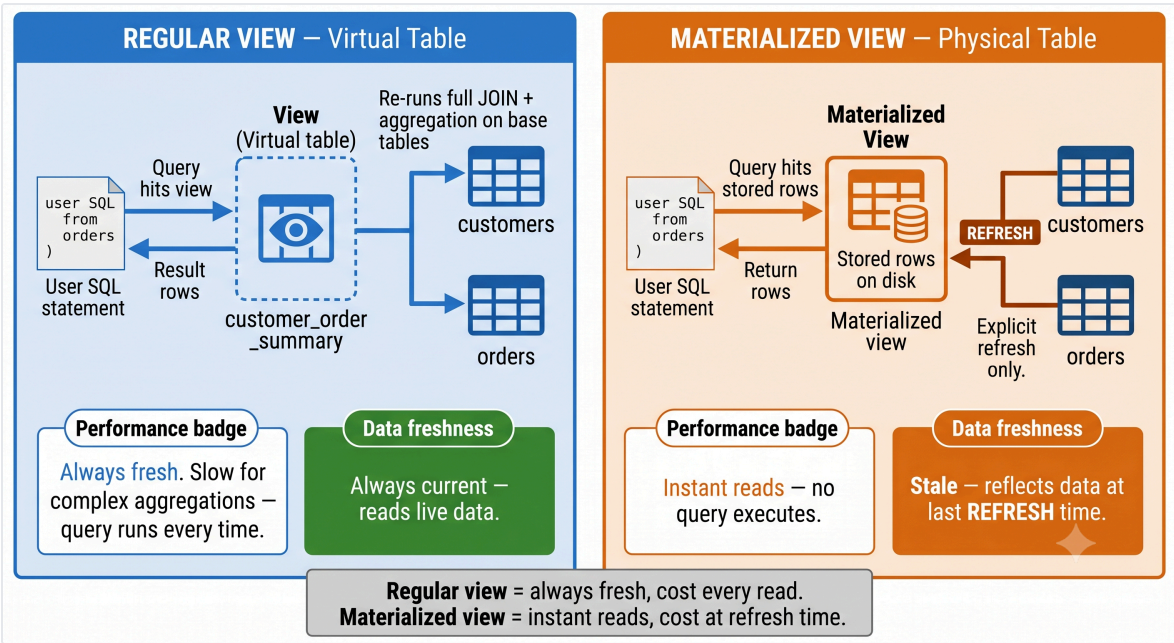
Student

What schema are we using?

CONCEPT CARD

Same e-commerce schema:

- `customers(customer_id, name, email, phone, city, signup_date, tier, is_active)`
- `orders(order_id, customer_id, order_date, total_amount, status, region)`
- `products(product_id, name, category_id, price, stock_qty)`
- `order_items(order_id, product_id, quantity, unit_price)`
- `regions(region_id, region_name, country)`



Regular View vs Materialized View — Core Difference

Regular View vs Materialized View — Quick Reference

Property	Regular View	Materialized View
Stores data on disk?	No — virtual table, no rows stored	Yes — result rows physically stored
Read performance	Executes full query on every read	Instant — reads pre-stored rows
Data freshness	Always current — reads live data	Stale until explicitly refreshed

Property	Regular View	Materialized View
Refresh required?	Never — always runs live	Yes — manual or scheduled REFRESH
Storage cost	Zero — no rows stored	Proportional to result set size
Index support	No — cannot index a virtual table	Yes — can CREATE INDEX on materialized view
Updatable?	Sometimes — strict conditions only	No — read-only
Snowflake support	Yes — standard CREATE VIEW	Yes — with automatic background refresh
Best for	Encapsulation, security, simple joins	Expensive aggregations powering dashboards
dbt equivalent	dbt view materialization	dbt table or incremental materialization

sql – View and Materialized View Syntax Templates

```
-- Regular VIEW
CREATE OR REPLACE VIEW view_name AS
SELECT col1, col2, ...
FROM   table1
JOIN   table2 ON ...
WHERE  condition;

-- Query a view exactly like a table
SELECT * FROM view_name WHERE col1 = 'value';

-- MATERIALIZED VIEW
CREATE MATERIALIZED VIEW matview_name AS
SELECT col1, col2, ...
FROM   table1
JOIN   table2 ON ...
GROUP BY col1;

-- Refresh options (PostgreSQL)
REFRESH MATERIALIZED VIEW matview_name;           -- locks during
refresh
```

```

REFRESH MATERIALIZED VIEW CONCURRENTLY matview_name; -- no lock, needs
UNIQUE index

-- Index on materialized view (for fast reads)
CREATE INDEX idx_matview_col ON matview_name(col1);

-- Drop views
DROP VIEW view_name;
DROP MATERIALIZED VIEW matview_name;

```

REGULAR VIEWS — VIRTUAL TABLES FOR ENCAPSULATION AND REUSE

ChatGPT • AI Assistant

A three-step execution flow diagram. Step 1 (top, blue box): User writes 'SELECT customer_name, lifetime_spend FROM customer_order_summary WHERE city = Delhi AND total_orders > 5.' Step 2 (middle, teal box, label 'View Expansion + Predicate Pushdown'): The view definition is substituted inline. An annotation arrow labeled 'Optimizer pushes WHERE city=Delhi into base table scan' points down to the customers table icon. The customers table shows a filter applied before the JOIN reducing rows from 500K to 25K. Step 3 (bottom, green box): The expanded query runs: 25K Delhi customers JOINed to orders table, GROUP BY applied, HAVING filter on total_orders > 5. Result: small filtered result set returned. A performance note reads: 'Predicate pushdown: filter before JOIN = far fewer rows to join. Without pushdown: join 500K rows then filter = expensive.' Clean three-step layout, arrows showing data flow, annotation highlighting the pushdown optimization.

View Query Execution — Predicate Pushdown

sql – Creating and Using Views – Encapsulation, Security, Maintainability

```

-- Create the customer order summary view
CREATE OR REPLACE VIEW customer_order_summary AS
SELECT
    c.customer_id,
    c.name                                     AS customer_name,
    c.city,
    c.tier,
    COUNT(o.order_id)                         AS total_orders,
    COALESCE(SUM(o.total_amount), 0)          AS lifetime_spend,
    COALESCE(AVG(o.total_amount), 0)          AS avg_order_value,
    MAX(o.order_date)                         AS last_order_date,
    MIN(o.order_date)                         AS first_order_date
FROM    customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city, c.tier;

```

```
-- Query the view like a table (no JOIN knowledge needed)
SELECT customer_name, city, lifetime_spend, tier
FROM   customer_order_summary
ORDER BY lifetime_spend DESC LIMIT 20;

-- Filter the view (predicate pushed into base table scan)
SELECT customer_name, total_orders, last_order_date
FROM   customer_order_summary
WHERE  city = 'Delhi' AND total_orders > 5;

-- SECURITY VIEW: hide sensitive columns
CREATE OR REPLACE VIEW customers_public AS
SELECT customer_id, name, city, tier, signup_date
FROM   customers;
-- email, phone, national_id are excluded

-- Grant view access, revoke base table access
GRANT SELECT ON customers_public TO analyst_role;
REVOKE SELECT ON customers          FROM analyst_role;

-- Verify what the view returns
SELECT * FROM customers_public LIMIT 5;

-- View metadata (PostgreSQL)
SELECT viewname, definition
FROM   pg_views
WHERE  schemaname = 'public'
      AND viewname = 'customer_order_summary';
```

Student

Let us start with regular views. Show me the customer order summary example — the view that hides the join complexity.

ChatGPT • AI Assistant

CONCEPT CARD

A **view** is created with **CREATE VIEW** and stores the SELECT definition, not the data. Every query against the view re-executes the stored SELECT, substituting the view definition inline. From the outside, it looks and behaves exactly like a table.

EXAMPLE CARD

Example 1 — Creating the customer_order_summary view:

sql – Code Example

```
CREATE OR REPLACE VIEW customer_order_summary AS
SELECT
    c.customer_id,
    c.name                                AS customer_name,
    c.city,
    c.tier,
    COUNT(o.order_id)                    AS total_orders,
    COALESCE(SUM(o.total_amount), 0)      AS lifetime_spend,
    COALESCE(AVG(o.total_amount), 0)      AS avg_order_value,
    MAX(o.order_date)                    AS last_order_date,
    MIN(o.order_date)                    AS first_order_date,
    SUM(CASE WHEN o.status = 'delivered'
           THEN o.total_amount ELSE 0 END) AS delivered_spend
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city, c.tier;
```

Once created, downstream users query it exactly like a table — they never need to know about the JOIN or GROUP BY:

sql – Code Example

```
-- Analyst queries: simple SELECT against the view
SELECT customer_name, city, lifetime_spend, tier
FROM   customer_order_summary
ORDER BY lifetime_spend DESC
LIMIT 20;

-- Filter by city – the WHERE is applied on top of the view
SELECT customer_name, total_orders, last_order_date
FROM   customer_order_summary
WHERE  city = 'Delhi'
      AND total_orders > 5;

-- Join the view to another table
SELECT cos.customer_name,
       cos.lifetime_spend,
       cos.tier,
       r.region_name
FROM   customer_order_summary cos
```

```
JOIN    customers c ON cos.customer_id = c.customer_id
JOIN    regions r   ON c.city = r.city;
```

Student

What actually happens when I query the view? Does the database store the join result?

ChatGPT • AI Assistant

No storage at all. When you write `SELECT * FROM customer_order_summary WHERE city = 'Delhi'`, the database substitutes the view definition inline and executes:

sql – Code Example

```
-- What the database actually runs (view expanded):
SELECT customer_name, total_orders, last_order_date
FROM (
    SELECT c.customer_id, c.name AS customer_name, c.city, ...
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name, c.city, c.tier
) view_expansion
WHERE city = 'Delhi'
AND total_orders > 5;
```

The query optimizer often pushes the WHERE condition down into the view query — in many cases `WHERE city = 'Delhi'` is pushed into the base table scan, filtering before the JOIN. This is called **predicate pushdown**.

CONCEPT CARD

Three reasons views are used everywhere:

1. Encapsulation — hide complexity

sql – Code Example

```
-- Without view: every report writer must know this join
SELECT c.name, SUM(o.total_amount), MAX(o.order_date)
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;

-- With view: one line per report
```

```
SELECT customer_name, lifetime_spend, last_order_date
FROM customer_order_summary;
```

2. Security — expose only safe columns

sql — Code Example

```
-- Base table has sensitive columns: email, phone, national_id
-- Create a view that exposes ONLY safe columns:
CREATE VIEW customers_public AS
SELECT customer_id, name, city, tier, signup_date
FROM customers;
-- Do NOT include: email, phone, national_id

-- Grant access to the VIEW only, not the base table:
GRANT SELECT ON customers_public TO analyst_role;
REVOKE SELECT ON customers FROM analyst_role; -- block base table
access
```

3. Maintainability — fix in one place

sql — Code Example

```
-- If the join condition changes, update the view once:
CREATE OR REPLACE VIEW customer_order_summary AS
-- ... updated definition ...
-- All 50 reports and dashboards that query this view
-- automatically use the new logic on next execution
```

WARNING

Views do not improve performance on their own. A complex view with 5 joins still executes those 5 joins on every query. If you find a view is slow, the fix is not to redesign the view — it is to add indexes to the base tables, or to convert to a materialized view.

INTERVIEW CARD

Frequency: High

Question: What is a view in SQL? What are its main use cases?

Expected Answer: A view is a named, stored SELECT query — a virtual table that stores no data. Every query against a view re-executes the underlying SELECT on the base tables. Main use cases: encapsulation — hiding complex join and aggregation logic behind a simple table name; security — exposing only specific columns to users

without granting access to base tables; and maintainability — when the underlying logic changes, updating the view definition propagates to all consumers automatically. Key limitation: views do not cache results, so complex view queries can be slow on large tables.

Follow-up: What is predicate pushdown in the context of views? (Answer: When you add a WHERE clause to a query on a view, the optimizer often 'pushes' that filter condition down into the base table scan inside the view definition — filtering rows before the JOIN executes rather than after. This makes filtered view queries much faster than filtering the full join result.)

🔗 PRACTICE QUIZ

1. Does a regular view store data on disk?
2. If you update the base table after creating a view, does the view show the updated data?
3. What is predicate pushdown and why does it matter for view performance?
4. You have a view joining 4 tables with GROUP BY. A query against this view takes 30 seconds. Changing the view definition will fix this — true or false?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. No — a view stores only the SELECT definition. No rows are stored. Every query re-executes the SELECT on the live base tables.
2. Yes — because the view re-executes its SELECT against the live base tables on every query, it always reflects the current state of the data.
3. Predicate pushdown: when you filter a view with WHERE, the optimizer pushes the filter into the base table scan rather than filtering after the full JOIN result is produced. This significantly reduces the rows that must be joined. It matters because it means `SELECT * FROM view WHERE city = 'Delhi'` can be nearly as fast as `SELECT * FROM customers WHERE city = 'Delhi'` if the optimizer pushes the filter down.
4. False — the view definition is just a saved SELECT. Changing it to a different SELECT does not change the query execution performance unless you simplify the logic or fix a structural issue. To fix performance, add indexes to base tables or convert to a materialized view.

🔗 CHECKPOINT QUESTION

A view `daily_revenue_report` joins orders, customers, orderitems, and products with `GROUP BY` region and orderdate, computing `SUM(revenue)` per day per region. It takes 45 seconds to query. A product manager queries it every 5 minutes from their BI tool. Diagnose the problem and suggest two different solutions with their trade-offs.

UPDATABLE VIEWS AND THEIR STRICT LIMITATIONS

ChatGPT • AI Assistant

A checklist diagram with two columns. Left column (green background, label 'UPDATABLE — all conditions met'): a view definition showing 'SELECT customer_id, name, city FROM customers WHERE is_active = TRUE.' Below, a checklist with green checkmarks: Single base table ✓, No GROUP BY ✓, No aggregates ✓, No DISTINCT ✓, No JOINS ✓, No window functions ✓. Result badge: 'UPDATE/INSERT/DELETE allowed.' Right column (red background, label 'NOT UPDATABLE — conditions violated'): a view definition showing 'SELECT c.name, COUNT(o.order_id) FROM customers c JOIN orders o ON ... GROUP BY c.name.' Below, a checklist with red X marks: Multiple tables (JOIN) X, GROUP BY X, Aggregate function X. Result badge: 'READ ONLY — no DML operations.' Below both columns, a WITH CHECK OPTION box reads: 'WITH CHECK OPTION — add to updatable views to prevent INSERT/UPDATE creating rows that disappear from the view filter.' Clean two-column layout, checklist icons, consistent color scheme.

Updatable vs Non-Updatable Views — Decision Checklist

sql – Updatable Views, WITH CHECK OPTION, and Non-Updatable Examples

```
-- UPDATABLE: single table, no aggregation
CREATE OR REPLACE VIEW active_customers AS
SELECT customer_id, name, city, tier, signup_date
FROM customers
WHERE is_active = TRUE;

-- UPDATE through the view works
UPDATE active_customers SET city = 'Mumbai' WHERE customer_id = 101;

-- INSERT WITHOUT CHECK OPTION: new row disappears from view!
INSERT INTO active_customers (name, city, tier)
VALUES ('Ghost User', 'Delhi', 'Bronze');
-- Inserted with is_active = NULL → not visible in active_customers!

-- Fix with WITH CHECK OPTION
CREATE OR REPLACE VIEW active_customers AS
SELECT customer_id, name, city, tier, signup_date
FROM customers
WHERE is_active = TRUE
WITH CHECK OPTION;

-- Now this INSERT is REJECTED:
```

```
INSERT INTO active_customers (name, city, tier)
VALUES ('Ghost User', 'Delhi', 'Bronze');
-- ERROR: new row violates check option for view 'active_customers'

-- This INSERT is ACCEPTED (is_active provided explicitly):
INSERT INTO customers (name, city, tier, is_active)
VALUES ('Real User', 'Delhi', 'Bronze', TRUE);
-- Insert through base table directly – better practice

-- NON-UPDATABLE: JOIN present
CREATE VIEW order_details_view AS
SELECT o.order_id, c.name, o.total_amount
FROM   orders o JOIN customers c ON o.customer_id = c.customer_id;
-- UPDATE order_details_view SET total_amount = 500 WHERE order_id = 1;
-- ERROR: cannot update a view based on multiple tables

-- NON-UPDATABLE: GROUP BY present
CREATE VIEW city_order_summary AS
SELECT city, COUNT(*) AS order_count, SUM(total_amount) AS revenue
FROM   orders o JOIN customers c ON o.customer_id = c.customer_id
GROUP BY city;
-- Read-only: collapsed rows cannot be traced back to individual orders
```

Student

Can I INSERT or UPDATE rows through a view?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — but only under very strict conditions. A view is **updatable** only if the database can unambiguously trace every change back to exactly one base table row. The moment any complexity enters the view definition, it becomes non-updatable.

A view is updatable if it has ALL of the following:

- Maps to exactly one base table (no JOINS)
- No aggregations (no GROUP BY, no COUNT, SUM, AVG, etc.)
- No DISTINCT
- No window functions
- No subqueries in the SELECT list

- The target column is not computed or derived

EXAMPLE CARD**Example 1 — Updatable view (simple, single-table):**

sql – Code Example

```
-- This view IS updatable
CREATE VIEW active_customers AS
SELECT customer_id, name, city, tier
FROM   customers
WHERE  is_active = TRUE;

-- INSERT through the view (inserts into customers)
INSERT INTO active_customers (name, city, tier)
VALUES ('New Customer', 'Delhi', 'Bronze');
-- This inserts into the customers table
-- BUT: is_active will be NULL – not set to TRUE!

-- UPDATE through the view
UPDATE active_customers
SET    city = 'Mumbai'
WHERE  customer_id = 101;
-- Works: maps cleanly to customers.city
```

⚠ WARNING

The INSERT gotcha — when you insert through a view with a WHERE clause, the new row may immediately disappear from the view:

sql – Code Example

```
-- View shows only active customers
CREATE VIEW active_customers AS
SELECT customer_id, name, city, tier
FROM   customers WHERE is_active = TRUE;

-- Insert a row: is_active defaults to NULL (not TRUE)
INSERT INTO active_customers (name, city) VALUES ('Ghost', 'Delhi');
-- Row inserted into customers with is_active = NULL
-- is_active = NULL fails WHERE is_active = TRUE
-- → The new row is NOT visible in active_customers immediately!
```

Fix this with **WITH CHECK OPTION** :

sql – Code Example

```
CREATE OR REPLACE VIEW active_customers AS
SELECT customer_id, name, city, tier
FROM   customers
WHERE  is_active = TRUE
WITH CHECK OPTION;
-- Now any INSERT or UPDATE that would make the row invisible
-- in the view is REJECTED with an error.
-- Forces is_active = TRUE on all inserts through this view.
```

Example 2 — Non-updatable views (most real-world views):

sql – Code Example

```
-- NOT updatable: has a JOIN
CREATE VIEW customer_order_summary AS
SELECT c.name, COUNT(o.order_id) AS orders
FROM   customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
-- UPDATE customer_order_summary SET orders = 5 WHERE name = 'Amelia';
-- ERROR: cannot update a view with GROUP BY and multiple tables

-- NOT updatable: has GROUP BY
CREATE VIEW city_revenue AS
SELECT city, SUM(total_amount) AS revenue
FROM   orders GROUP BY city;
-- Which row in orders would 'UPDATE city_revenue SET revenue = 1000'
update?
-- Impossible — the view collapsed many rows into one.
```

Student

In practice, do data engineers ever update through views?

ChatGPT • AI Assistant

Rarely — and with caution. In data engineering, views are almost always read-only. The main use case for updatable views is application layer security: you give an application a view and it can only modify rows that pass the view's WHERE condition. For data engineering pipelines, you update the base tables directly and read through views.

 INTERVIEW CARD

Frequency: Medium

Question: What are the conditions that make a view updatable?

Expected Answer: A view is updatable only when it maps unambiguously to a single base table with no complexity: no JOINS, no GROUP BY, no aggregate functions, no DISTINCT, no window functions, and no computed columns. Any of these makes it impossible for the database to trace an INSERT, UPDATE, or DELETE back to a single base table row. WITH CHECK OPTION adds protection: any row inserted or updated through the view must satisfy the view's WHERE clause — if it would become invisible in the view, the operation is rejected.

Follow-up: What problem does WITH CHECK OPTION solve? (Answer: Without it, you can INSERT a row through a filtered view that does not satisfy the view's WHERE clause — the row is inserted but immediately disappears from the view. WITH CHECK OPTION rejects any write that would create a row invisible in the view.)

 PRACTICE QUIZ

1. Can you update a view that has a GROUP BY clause? Why or why not?
2. You INSERT into a view with `WHERE is_active = TRUE` without WITH CHECK OPTION. What happens to the new row?
3. What is WITH CHECK OPTION?
4. Name three view properties that make it non-updatable.

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

 ANSWER KEY

1. No — GROUP BY collapses multiple rows into one aggregate row. There is no way to trace an UPDATE or INSERT on that aggregate row back to individual base table rows unambiguously.
2. The row is inserted into the base table with `is_active = NULL` (or whatever default the base table has). Since `NULL ≠ TRUE`, the row does not appear in the view — it 'disappears' immediately after insertion.
3. WITH CHECK OPTION is a constraint on updatable views that rejects any INSERT or UPDATE that would make the row invisible in the view — i.e., the new row must satisfy the view's WHERE clause.

4. Any three of: JOIN (multiple tables), GROUP BY, aggregate function (COUNT/SUM/AVG), DISTINCT, window function, subquery in SELECT list, computed/derived column.

🔍 CHECKPOINT QUESTION

You create this view: `CREATE VIEW premium_orders AS SELECT order_id, customer_id, total_amount FROM orders WHERE total_amount ≥ 1000; —` without WITH CHECK OPTION. Explain what happens when you run `INSERT INTO premium_orders (customer_id, total_amount) VALUES (101, 500);` and what happens when you run `UPDATE premium_orders SET total_amount = 800 WHERE order_id = 1001;` after adding WITH CHECK OPTION.

MATERIALIZED VIEWS — PHYSICAL STORAGE, REFRESH STRATEGIES, AND INDEXES

ChatGPT • AI Assistant

A two-panel side-by-side diagram. Left panel (red background, label 'REFRESH — Blocking'): Timeline shows T=0: REFRESH starts. T=0 to T=45s: Lock icon over materialized view labeled 'View LOCKED — all queries block.' T=45s: New data written, lock released. Dashboard query at T=20s shown with a wait icon labeled 'Waiting 25 more seconds.' Performance badge: 'Downtime: 45 seconds per refresh. Simple, no index required.' Right panel (green background, label 'REFRESH CONCURRENTLY — Non-Blocking'): Timeline shows T=0: CONCURRENTLY refresh starts in shadow. T=0 to T=60s: View remains readable — dashboard queries run against old data, labeled 'Reads old data (stale but available).' T=60s: Diff computed and applied, view updated. Dashboard query at T=20s shown with a green checkmark labeled 'Returns instantly (slightly stale).' Performance badge: 'Zero downtime. Requires UNIQUE index. Slower refresh.' A rule box below: 'Production dashboards → CONCURRENTLY. Nightly batch → blocking REFRESH is fine.' Clean timeline-based two-panel layout, consistent color coding.

Materialized View Refresh Strategies — Blocking vs Concurrent

sql — Materialized View — Create, Index, Refresh Strategies, Airflow Scheduling

```
-- Create materialized view (data populated on creation)
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS
SELECT
    o.region,
    DATE_TRUNC('day', o.order_date)           AS order_day,
    COUNT(DISTINCT o.order_id)                AS total_orders,
    COUNT(DISTINCT o.customer_id)            AS unique_customers,
    SUM(o.total_amount)                      AS total_revenue,
    AVG(o.total_amount)                      AS avg_order_value
FROM    orders o
WHERE   o.status ≠ 'cancelled'
GROUP BY o.region, DATE_TRUNC('day', o.order_date);
```

```
-- Add index for fast filtered reads
CREATE INDEX idx_mv_region_day
    ON mv_daily_revenue_by_region(region, order_day);

-- Add UNIQUE index for CONCURRENTLY refresh
CREATE UNIQUE INDEX idx_mv_region_day_unique
    ON mv_daily_revenue_by_region(region, order_day);

-- BLOCKING refresh (simple, locks during refresh)
REFRESH MATERIALIZED VIEW mv_daily_revenue_by_region;

-- NON-BLOCKING refresh (needs unique index, zero downtime)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue_by_region;

-- Check freshness
SELECT matviewname, last_refresh, is_populated
FROM    pg_matviews
WHERE   matviewname = 'mv_daily_revenue_by_region';

-- Query the materialized view (instant reads)
SELECT * FROM mv_daily_revenue_by_region
WHERE   region = 'South'
    AND  order_day ≥ '2024-01-01'
ORDER BY order_day DESC;

-- Compare: regular view equivalent (runs full query each time)
CREATE OR REPLACE VIEW v_daily_revenue_by_region AS
SELECT o.region, DATE_TRUNC('day', o.order_date) AS order_day,
       COUNT(*) AS total_orders, SUM(total_amount) AS total_revenue
FROM orders o WHERE status ≠ 'cancelled'
GROUP BY o.region, DATE_TRUNC('day', o.order_date);
-- Querying this takes 2-4 seconds each time (vs <5ms from materialized view)
```

Student

Now materialized views. You said they store the actual data. Show me how to create one and how refresh works.

CONCEPT CARD

A **materialized view** physically stores the query result as rows on disk. When you query it, the database reads pre-stored rows — just like a regular table — with no JOIN or aggregation executing at read time. The trade-off is that the stored data goes stale when the source tables change.

EXAMPLE CARD**Example 1 — Create a materialized view for daily revenue by region:**

sql – Code Example

```
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS
SELECT
    o.region,
    DATE_TRUNC('day', o.order_date)           AS order_day,
    COUNT(DISTINCT o.order_id)                AS total_orders,
    COUNT(DISTINCT o.customer_id)            AS unique_customers,
    SUM(o.total_amount)                       AS total_revenue,
    AVG(o.total_amount)                       AS avg_order_value
FROM    orders o
WHERE   o.status ≠ 'cancelled'
GROUP BY o.region, DATE_TRUNC('day', o.order_date)
ORDER BY order_day DESC, total_revenue DESC;

-- This runs the full query ONCE and stores the result on disk.
-- Subsequent reads are instant – no GROUP BY, no JOIN executes.
```

Why this is faster than a regular view:

sql – Code Example

```
-- Regular view: each dashboard request executes:
-- → Full scan of 10M orders table
-- → GROUP BY region + date
-- → 2-4 seconds per dashboard load

-- Materialized view: each dashboard request reads:
-- → Pre-stored rows: ~365 rows per region per year
-- → < 5 milliseconds per dashboard load
-- → 400x to 800x faster for a year of data
```

Adding an index to a materialized view:

sql – Code Example

```
-- Materialized views support indexes – unlike regular views
CREATE INDEX idx_mv_region_day
  ON mv_daily_revenue_by_region(region, order_day);

-- Now even filtered queries are instant:
SELECT * FROM mv_daily_revenue_by_region
WHERE region = 'South' AND order_day ≥ '2024-01-01';
-- Index lookup on stored rows: < 1ms
```

Student

What does REFRESH do and how do I know when to run it?

ChatGPT • AI Assistant

CONCEPT CARD

Two REFRESH strategies:**Strategy 1 — REFRESH MATERIALIZED VIEW (blocking)**

sql – Code Example

```
REFRESH MATERIALIZED VIEW mv_daily_revenue_by_region;
```

- Drops the old stored data, re-runs the full SELECT, stores new results.
- **Locks the materialized view during refresh** — queries against the view block until refresh completes.
- If the underlying query takes 30 seconds, the view is unavailable for 30 seconds.
- Safe for low-traffic or overnight batch scenarios.

Strategy 2 — REFRESH MATERIALIZED VIEW CONCURRENTLY (non-blocking)

sql – Code Example

```
-- Requirement: a UNIQUE index must exist on the materialized view
CREATE UNIQUE INDEX idx_mv_region_day_unique
  ON mv_daily_revenue_by_region(region, order_day);

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue_by_region;
```

- Runs the full SELECT in a shadow copy, computes the diff, applies changes.
- **Zero downtime** — existing queries read the old data while refresh runs.
- **Requires a UNIQUE index** on the materialized view — PostgreSQL uses this to compute the diff efficiently.
- Slower than blocking refresh (computes a diff instead of full replace), but ideal for production dashboards.

Scheduled refresh using Airflow:

python — Code Example

```
# Airflow DAG: refresh materialized view every hour
from airflow import DAG
from airflow.providers.postgres.operators.postgres import PostgresOperator
from datetime import datetime, timedelta

with DAG('refresh_mv_daily_revenue',
         schedule_interval='@hourly',
         start_date=datetime(2024, 1, 1)) as dag:

    refresh_task = PostgresOperator(
        task_id='refresh_mv',
        postgres_conn_id='warehouse_db',
        sql='REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_revenue_by_region;'
    )
```

Scheduled refresh using cron (simpler):

bash — Code Example

```
# crontab: refresh every hour at :05
5 * * * * psql -c "REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_revenue_by_region;"
```

Student

How do I know how stale my materialized view is?

Check the last refresh time in PostgreSQL system tables:

sql – Code Example

```
SELECT schemaname,  
       matviewname,  
       last_refresh,  
       is_populated  
FROM   pg_matviews  
WHERE  matviewname = 'mv_daily_revenue_by_region';
```

❗ COMMON PITFALL / MISTAKE

✗ Querying a freshly created but unpopulated materialized view:

sql – Code Example

```
CREATE MATERIALIZED VIEW mv_test AS  
SELECT * FROM orders;  
-- The view is created BUT data is loaded during creation.  
-- If created WITH NO DATA:  
CREATE MATERIALIZED VIEW mv_test AS SELECT * FROM orders WITH NO DATA;  
-- mv_test is empty – any query returns zero rows!  
-- Must run REFRESH before querying.
```

✅ Correct — populate immediately or check `is\_populated`:

sql – Code Example

```
-- Default: data is populated on CREATE  
CREATE MATERIALIZED VIEW mv_test AS SELECT * FROM orders;  
-- Or explicitly refresh after WITH NO DATA:  
REFRESH MATERIALIZED VIEW mv_test;
```

📌 INTERVIEW CARD

Frequency: High

Question: What is the difference between REFRESH MATERIALIZED VIEW and REFRESH MATERIALIZED VIEW CONCURRENTLY?

Expected Answer: REFRESH MATERIALIZED VIEW is a blocking refresh — it drops the existing stored data and re-runs the full SELECT to generate new results. During refresh, queries against the materialized view are blocked until it completes.

REFRESH MATERIALIZED VIEW CONCURRENTLY is non-blocking — it runs the SELECT in parallel, computes a diff, and applies only the changed rows. Existing queries read the old data while refresh runs. CONCURRENTLY requires a UNIQUE index on the materialized view so PostgreSQL can efficiently compute which rows changed. CONCURRENTLY is slower than blocking refresh but essential for production dashboards where zero downtime is required.

Follow-up: If a REFRESH MATERIALIZED VIEW CONCURRENTLY requires a UNIQUE index, what should the unique key be for `mv_daily_revenue_by_region` ? (Answer: The combination (region, orderday) — *the GROUP BY columns uniquely identify each row in the materialized view. Create: 'CREATE UNIQUE INDEX ON mvdailyrevenuebyregion(region, orderday);'*)

🔗 PRACTICE QUIZ

1. What happens to queries against a materialized view during a blocking REFRESH?
2. What does REFRESH MATERIALIZED VIEW CONCURRENTLY require to work?
3. How do you check when a materialized view was last refreshed in PostgreSQL?
4. Predict: you `CREATE MATERIALIZED VIEW mvtest AS SELECT * FROM orders WITH NO DATA`. Then `SELECT COUNT(*) FROM mvtest`. What is the result?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. Queries are blocked — the view is locked during the full re-computation. They wait until the refresh completes.
2. A UNIQUE index on the materialized view — PostgreSQL uses it to compute the diff between old and new results.
3. `SELECT last_refresh, is_populated FROM pg_matviews WHERE matviewname = 'mv_test';`
4. Zero rows — WITH NO DATA creates the materialized view empty without running the SELECT. You must run REFRESH MATERIALIZED VIEW mv_test before querying.

🔗 CHECKPOINT QUESTION

Your `mv_daily_revenue_by_region` materialized view takes 45 seconds to refresh. Your dashboard refreshes every 5 minutes. You need to ensure the dashboard never shows a 'loading' state. Write the complete strategy: which REFRESH type, what index to create first, and the Airflow task configuration.

SNOWFLAKE MATERIALIZED VIEWS — AUTO-REFRESH, CLUSTERING, AND COST

ChatGPT • AI Assistant

A two-part diagram. Top half (label 'Automatic Refresh Architecture'): shows three elements connected by arrows. Base table 'orders' with a continuous INSERT stream icon. A background refresh service box labeled 'Snowflake MV Refresh Service — detects base table changes, triggers refresh.' Materialized view icon labeled 'mv_daily_revenue_by_region.' Arrow from orders to refresh service: 'Change detected.' Arrow from refresh service to MV: 'Background refresh — credits consumed.' Dashboard query arrow hits MV directly: 'Instant read from stored rows.' Credit meter icon with warning: 'Monitor credits in MATERIALIZED_VIEW_REFRESH_HISTORY.' Bottom half (label 'CLUSTER BY Micro-Partition Pruning'): shows a grid of 12 micro-partition blocks labeled p1 through p12. Without clustering: all 12 blocks highlighted for a WHERE region=South query. With CLUSTER BY (region): only 3 blocks highlighted green (containing South data), 9 blocks grayed out with label 'Pruned — skipped.' Performance badge: 'With clustering: 75% scan reduction for region-filtered queries.' Clean two-part layout, consistent Snowflake color scheme (blue).

Snowflake Materialized View — Auto-Refresh and Micro-Partition Pruning

sql – Snowflake Materialized Views – Creation, Clustering, Cost Monitoring

```
-- Create Snowflake materialized view with clustering
CREATE OR REPLACE MATERIALIZED VIEW mv_daily_revenue_by_region
CLUSTER BY (region, order_date)
AS
SELECT
    region,
    order_date::DATE           AS order_date,
    COUNT(*)                   AS total_orders,
    SUM(total_amount)          AS total_revenue,
    AVG(total_amount)          AS avg_order_value
FROM    orders
WHERE   status ≠ 'cancelled'
GROUP BY region, order_date::DATE;

-- NO REFRESH needed – Snowflake handles it automatically

-- Transparent query rewrite: this query ...
SELECT region, order_date::DATE, SUM(total_amount)
FROM    orders WHERE status ≠ 'cancelled'
GROUP BY region, order_date::DATE;
-- ...is automatically rewritten to use the MV
```

```
-- Check MV status
SHOW MATERIALIZED VIEWS IN SCHEMA public;

-- Monitor refresh credits consumed
SELECT name, refresh_start_time, refresh_end_time,
       bytes_written, rows_written, credits_used
FROM   SNOWFLAKE.ACCOUNT_USAGE.MATERIALIZED_VIEW_REFRESH_HISTORY
WHERE  name ILIKE '%REVENUE%'
ORDER BY refresh_start_time DESC
LIMIT 20;

-- Disable automatic refresh if costs are too high
ALTER MATERIALIZED VIEW mv_daily_revenue_by_region SUSPEND;
ALTER MATERIALIZED VIEW mv_daily_revenue_by_region RESUME;

-- For high-insert tables: use a regular table + scheduled task instead
-- (avoids continuous refresh cost)
CREATE OR REPLACE TABLE daily_revenue_summary AS
SELECT region, order_date::DATE AS order_day,
       COUNT(*) AS total_orders, SUM(total_amount) AS total_revenue
FROM   orders WHERE status ≠ 'cancelled'
GROUP BY region, order_date::DATE;

-- Refresh via Snowflake task (scheduled, not continuous)
CREATE OR REPLACE TASK refresh_daily_revenue
  WAREHOUSE = compute_wh
  SCHEDULE  = 'USING CRON 0 * * * * UTC' -- every hour
AS
INSERT OVERWRITE INTO daily_revenue_summary
SELECT region, order_date::DATE,
       COUNT(*), SUM(total_amount)
FROM   orders WHERE status ≠ 'cancelled'
GROUP BY region, order_date::DATE;

ALTER TASK refresh_daily_revenue RESUME;
```

Student

This programme uses Snowflake. How are materialized views different there?

CONCEPT CARD

Snowflake materialized views work differently from PostgreSQL in three important ways:

1. **Automatic background refresh** — Snowflake detects changes to the base table and refreshes the materialized view automatically in the background. You do NOT run REFRESH commands manually.
2. **Billed separately** — automatic refresh consumes Snowflake credits on your credit meter. You pay for both storage (storing the materialized view rows) and compute (the background refresh service).
3. **SQL subset only** — Snowflake materialized views have a more restricted SQL subset than PostgreSQL. No JOINS, no external stages, no certain aggregations.

EXAMPLE CARD**Example 1 — Creating a Snowflake materialized view:**

sql – Code Example

```
-- Snowflake syntax
CREATE OR REPLACE MATERIALIZED VIEW mv_daily_revenue_by_region
CLUSTER BY (region, order_date) -- optional clustering for scan
pruning
AS
SELECT
    region,
    order_date::DATE           AS order_date,
    COUNT(*)                   AS total_orders,
    SUM(total_amount)          AS total_revenue
FROM    orders
WHERE   status ≠ 'cancelled'
GROUP BY region, order_date::DATE;
```

Key Snowflake differences:

sql – Code Example

```
-- Snowflake materialized views:
-- ✓ Automatic background refresh (no REFRESH command needed)
-- ✓ CLUSTER BY for micro-partition pruning
-- ✓ Queries can use the MV without knowing it exists (transparent
rewrite)
-- X No JOINS allowed in the MV definition
-- X Limited aggregate functions (no MEDIAN, PERCENTILE_CONT)
```

```
-- X Credits charged for automatic refresh service
-- X No CONCURRENTLY option (handled automatically)

-- Check materialized view status in Snowflake
SHOW MATERIALIZED VIEWS IN DATABASE my_db SCHEMA public;

-- See refresh history and credits used
SELECT *
FROM   SNOWFLAKE.ACCOUNT_USAGE.MATERIALIZED_VIEW_REFRESH_HISTORY
WHERE  name = 'MV_DAILY_REVENUE_BY_REGION'
ORDER BY refresh_start_time DESC
LIMIT 10;
```

Example 2 — Snowflake transparent query rewrite:

sql – Code Example

```
-- When Snowflake sees this query:
SELECT region, order_date::DATE, SUM(total_amount)
FROM   orders
WHERE  status ≠ 'cancelled'
GROUP BY region, order_date::DATE;

-- It automatically rewrites to use the materialized view:
-- → Reads from mv_daily_revenue_by_region (stored rows)
-- → No full table scan of orders
-- This happens TRANSPARENTLY – query doesn't mention the MV
```

Student

What is CLUSTER BY and why does it matter?

ChatGPT • AI Assistant

Clustering in Snowflake defines the sort order of micro-partitions — Snowflake's internal storage blocks (each ~16MB of compressed data). When you cluster by **(region, order_day)**, rows with the same region and nearby dates are stored in the same micro-partitions.

When you query **WHERE region = 'South' AND order_day ≥ '2024-01-01'**, Snowflake can skip entire micro-partitions that do not contain 'South' data — this is called **micro-partition pruning**.

sql – Code Example

```
-- Without clustering: query scans all micro-partitions
SELECT * FROM mv_daily_revenue_by_region WHERE region = 'South';
-- Scans: all partitions (100% of data)

-- With CLUSTER BY (region, order_day):
-- Scans: only partitions containing 'South' region data
-- Pruning: 80-95% of micro-partitions skipped depending on data
distribution
```

When to cluster:

- Large materialized views (> 1 billion rows in the base table)
- Consistently filtered by the same column(s) in dashboards
- Not needed for small MVs — clustering adds recluster cost

CONCEPT CARD**Snowflake materialized view cost considerations:**

code – Code Example

Cost 1: Storage

Materialized view rows are stored in Snowflake storage
Cost = rows × avg row size × Snowflake storage price
For mv_daily_revenue_by_region: 365 days × N regions = small

Cost 2: Automatic refresh credits

Every time the base orders table changes, Snowflake triggers a background refresh. For high-insert tables (orders arriving continuously), this can mean MANY refreshes per hour.

Monitor: MATERIALIZED_VIEW_REFRESH_HISTORY

Cost 3: Clustering maintenance

If CLUSTER BY is set, Snowflake runs automatic recluster (AUTO_CLUSTERING) to keep data sorted. This uses compute credits.

Rule of thumb: materialized views are cost-effective when:

- The base query saves more compute than the refresh costs
- The MV is queried frequently (100+ times per hour)
- The base table is NOT updated continuously (batch loads are cheaper)

**WARNING**

Do NOT create Snowflake materialized views on tables that receive continuous streaming inserts. Every micro-batch insertion triggers a background refresh cycle. On a table receiving 10,000 rows/second, this can generate enormous credit consumption. For streaming use cases, use a regular table or a zero-copy clone instead.

INTERVIEW CARD

Frequency: High (Snowflake-specific roles)

Question: How do Snowflake materialized views differ from PostgreSQL materialized views?

Expected Answer: Three key differences. First, Snowflake materialized views refresh automatically in the background when the base table changes — no REFRESH command needed, unlike PostgreSQL where you schedule REFRESH explicitly. Second, Snowflake charges credits for the automatic refresh service and for storage — cost monitoring is essential. Third, Snowflake supports CLUSTER BY on materialized views for micro-partition pruning, and supports transparent query rewrite — the optimizer automatically routes qualifying queries to the materialized view even when the query mentions the base table. Limitation: Snowflake materialized views do not support JOINS in the definition, unlike PostgreSQL.

Follow-up: What is micro-partition pruning and how does CLUSTER BY help? (Answer: Snowflake stores data in ~16MB compressed micro-partitions. Each partition records the min/max values of each column. CLUSTER BY ensures rows with similar column values are co-located, so queries with WHERE filters on the cluster key can skip entire partitions — scanning only relevant data.)

PRACTICE QUIZ

1. Do Snowflake materialized views need manual REFRESH commands?
2. What does CLUSTER BY do to a Snowflake materialized view?
3. Why are Snowflake materialized views potentially expensive on high-insert tables?
4. What does 'transparent query rewrite' mean in Snowflake?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. No — Snowflake automatically refreshes materialized views in the background when the base table changes. Manual REFRESH is a PostgreSQL concept.
2. CLUSTER BY defines the sort key for micro-partitions, enabling Snowflake to prune (skip) partitions that do not contain the queried values — dramatically reducing scan volume for filtered queries.
3. Every micro-batch insertion to the base table triggers a background refresh cycle, consuming compute credits. Continuous streaming inserts can trigger hundreds of refresh cycles per hour.
4. Transparent query rewrite: when a query's SELECT-GROUP BY pattern matches a materialized view definition, Snowflake automatically routes it to read from the materialized view instead of the base table — without the query mentioning the MV. This happens transparently at the optimizer level.

🔍 CHECKPOINT QUESTION

You have a Snowflake table `orders` receiving 5,000 new rows per minute from a Kafka pipeline. You want to create a materialized view `mv_hourly_revenue` aggregating revenue by hour and region. Identify the cost risk and propose an alternative architecture that achieves the same dashboard goal without triggering excessive refresh cycles.

CHOOSING BETWEEN VIEW AND MATERIALIZED VIEW — THE DECISION FRAMEWORK

ChatGPT • AI Assistant

A decision flowchart. Root diamond: 'Is the view query expensive (> 5 seconds)?' Left branch (NO): second diamond 'Is security or column masking the goal?' YES → green box 'Regular View — always fresh, zero storage, masks columns.' NO → green box 'Regular View with base table index — fast enough without MV.' Right branch (YES from root): third diamond 'Is real-time data required?' YES → orange box 'Fix indexes or partition base table — avoid views.' NO → fourth diamond 'How often is the view queried?' Less than 10 times/hour → orange box 'Regular View or scheduled table refresh — MV overhead not justified.' More than 10 times/hour → fifth diamond 'Snowflake or PostgreSQL?' Snowflake → green box 'Snowflake Materialized View with CLUSTER BY — auto-refresh, monitor credits.' PostgreSQL → green box 'Materialized View + REFRESH CONCURRENTLY + UNIQUE index + Airflow schedule.' Below all paths, a dbt strip reads: 'In a dbt stack: view → materialized: view. MV → materialized: table or incremental.' Clean flowchart, decision diamonds, green/orange/red terminal boxes.

View vs Materialized View Decision Framework

sql — Decision Framework Applied — Four Real Scenarios

```
-- SCENARIO 1: BI dashboard — daily revenue, 1-hour staleness OK
-- Decision: Materialized View + CONCURRENTLY refresh
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS
```

```

SELECT region, DATE_TRUNC('day', order_date) AS order_day,
       COUNT(*) AS orders, SUM(total_amount) AS revenue,
       CURRENT_TIMESTAMP AS last_refreshed_at -- timestamp for
dashboard
FROM   orders WHERE status ≠ 'cancelled'
GROUP BY region, DATE_TRUNC('day', order_date);

CREATE UNIQUE INDEX idx_mv_rev_unique
  ON mv_daily_revenue_by_region(region, order_day);

-- Airflow: REFRESH CONCURRENTLY every hour
-- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue_by_region;

-- SCENARIO 2: real-time order status
-- Decision: no view – query base table with index
CREATE INDEX idx_orders_customer_status ON orders(customer_id, status);
SELECT * FROM orders WHERE customer_id = 101 AND status = 'pending'; --
<5ms

-- SCENARIO 3: analyst access – no PII
-- Decision: Regular security view
CREATE OR REPLACE VIEW customers_analyst_safe AS
SELECT customer_id, name, city, tier, signup_date
FROM   customers;
-- email, phone, national_id excluded
GRANT SELECT ON customers_analyst_safe TO analyst_role;
REVOKE SELECT ON customers FROM analyst_role;

-- SCENARIO 4: monthly product performance – 120s query, 24h staleness
-- Decision: Materialized View refreshed once daily
CREATE MATERIALIZED VIEW mv_monthly_category_performance AS
SELECT p.category_id,
       cat.name AS
category_name,
       DATE_TRUNC('month', o.order_date) AS order_month,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit_price) AS gross_revenue
FROM   orders o
JOIN   order_items oi ON o.order_id = oi.order_id
JOIN   products p     ON oi.product_id = p.product_id
JOIN   categories cat ON p.category_id = cat.category_id
WHERE  o.status = 'delivered'
GROUP BY p.category_id, cat.name, DATE_TRUNC('month', o.order_date);

-- Refresh once daily at 3am (blocking is fine – low traffic overnight)

```

```
-- 0 3 * * * psql -c "REFRESH MATERIALIZED VIEW
mv_monthly_category_performance;"

-- dbt equivalents:
-- Scenario 1: models/marts/mv_daily_revenue.sql  {{
config(materialized='table') }}
-- Scenario 3: models/staging/customers_safe.sql  {{
config(materialized='view') }}
```

Student

When do I use a regular view vs a materialized view? Is there a clear rule?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — the decision comes down to two questions:

- 1. **How expensive is the underlying query?** If it takes < 1 second, a regular view is fine. If it takes > 5 seconds, a materialized view is justified.
- 2. **How stale can the data be?** If data must be real-time (live order status), use a regular view. If data can be 1-24 hours old (daily revenue dashboard), a materialized view is appropriate.

CONCEPT CARD

The decision framework:

code – Code Example		
Query execution time	Acceptable staleness	→ Recommended
< 1 second	Any	→ Regular view
1–5 seconds	Any	→ Regular view +
index on base tables		
> 5 seconds	Real-time required	→ Fix indexes,
avoid views entirely		
> 5 seconds	Hours acceptable	→ Materialized
view		
> 60 seconds	Daily acceptable	→ Materialized
view + partition base table		

Any	Security/encapsulation	→ Regular view
Any	Column masking only	→ Regular view

EXAMPLE CARD**Decision examples:****sql – Code Example**

```

-- CASE 1: live order status dashboard
-- Query: SELECT * FROM orders WHERE customer_id = ? AND status =
'pending'
-- Execution: < 10ms with index
-- Staleness: must be real-time (customer is waiting for updates)
-- Decision: NO VIEW needed – just index the base table

-- CASE 2: analyst portal – customer details
-- Query: SELECT customer_id, name, city, tier FROM customers
-- Staleness: acceptable to be hours old
-- Security need: hide email, phone from analysts
-- Decision: REGULAR VIEW (security masking)
CREATE VIEW customers_for_analysts AS
SELECT customer_id, name, city, tier, signup_date FROM customers;

-- CASE 3: BI dashboard – daily revenue by region (10M orders table)
-- Query: full scan + GROUP BY + 2-4 seconds execution
-- Staleness: acceptable to be up to 1 hour old (business analytics)
-- Decision: MATERIALIZED VIEW refreshed hourly
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS ... ;
-- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue_by_region;
-- Scheduled: every 60 minutes via Airflow

-- CASE 4: Snowflake dashboard on orders table with batch loads
-- Base table receives bulk loads once per hour (not streaming)
-- Query: complex multi-dimension aggregation
-- Decision: Snowflake MATERIALIZED VIEW with CLUSTER BY
-- Auto-refresh triggers once per hourly load – manageable cost

-- CASE 5: data quality report – joins 6 tables, runs nightly
-- Query: 3-minute execution
-- Staleness: daily refresh is fine (batch report)
-- Decision: MATERIALIZED VIEW refreshed once daily (blocking REFRESH
OK at 3am)

```

❗ COMMON PITFALL / MISTAKE

✗ Creating a materialized view for a simple, already-fast query:

sql – Code Example

```
-- BAD: this view query takes 5ms – materialized view is overkill
CREATE MATERIALIZED VIEW mv_active_customers AS
SELECT customer_id, name, city FROM customers WHERE is_active = TRUE;
-- Adds storage cost, refresh overhead, staleness risk
-- Regular view is better – always fresh, zero storage
```

✓ Use materialized view only for queries that are genuinely expensive:

sql – Code Example

```
-- GOOD: 10M rows, GROUP BY, complex aggregation – expensive query
CREATE MATERIALIZED VIEW mv_revenue_summary AS
SELECT region, category, DATE_TRUNC('month', order_date) AS month,
       COUNT(*) AS orders, SUM(total_amount) AS revenue
FROM   orders o
JOIN   order_items oi ON o.order_id = oi.order_id
JOIN   products p ON oi.product_id = p.product_id
GROUP BY region, category, DATE_TRUNC('month', order_date);
```

Student

What is the dbt equivalent of views and materialized views?

ChatGPT • AI Assistant

In dbt, the **materialization** setting controls how a model is stored:

yml – Code Example

```
# dbt_project.yml or model config
models:
  my_project:
    analytics:
      +materialized: view          # → CREATE VIEW
    summaries:
      +materialized: table        # → CREATE TABLE (like materialized
view)
```

facts:

+**materialized**: incremental # → INSERT/MERGE of new rows only

- **materialized: view** → same as a SQL view — always fresh, re-executes on each query.
- **materialized: table** → same as a materialized view — physical table, refreshed on **dbt run**.
- **materialized: incremental** → like a materialized view with incremental refresh — only processes new rows since last run.

For most data engineering work on Snowflake, **dbt models replace SQL views and materialized views** with better version control, testing, and documentation. Native SQL views and materialized views are used for: operational workflows, security masking, legacy compatibility, or cases where dbt is not in the stack.

 INTERVIEW CARD

Frequency: High

Question: When would you use a regular view versus a materialized view? Walk me through your decision framework.

Expected Answer: The two key factors are query cost and acceptable staleness. Regular views are appropriate when: the underlying query executes in under a second with proper indexes; security or column masking is the primary goal; data must always be real-time; or the use case is encapsulating join logic for downstream consumers. Materialized views are appropriate when: the query takes more than 5 seconds even with indexes (complex aggregations, many-table joins on large fact tables); the data can be acceptably stale by 1 hour to 24 hours; and the view is queried frequently enough that the refresh cost amortizes over many reads. In modern data stacks, dbt models (view or table materialization) replace native SQL views and materialized views for transformation logic, with native MVs reserved for security masking and Snowflake-specific performance optimization.

Follow-up: What is the staleness risk of a materialized view and how do you communicate it to BI users? (Answer: A materialized view reflects data as of the last REFRESH time. If new orders arrive after the last refresh, they are invisible in the view. Best practice: include a **last_refreshed_at** column or expose the refresh timestamp in the dashboard subtitle — 'Data as of: 14:05' — so users know the data age.)

 PRACTICE QUIZ

1. Your BI tool queries a view 500 times per hour. The underlying query takes 8 seconds. Should you convert to a materialized view?
2. A regular view hides sensitive columns. If you convert it to a materialized view, does the security benefit remain?
3. What dbt materialization is equivalent to a materialized view?
4. You want a materialized view to include a `last_refreshed_at` column. How would you do this?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. Yes — $500 \text{ queries} \times 8 \text{ seconds} = 4,000 \text{ seconds}$ of compute per hour. A materialized view refreshed once per hour replaces that with one 8-second refresh + 500 instant reads. The break-even is clearly in favour of the materialized view.
2. Yes — the materialized view stores the pre-computed result of the SELECT, which already excludes the sensitive columns. Users querying the MV never see email or phone.
3. `materialized: table` — dbt runs the SELECT and stores the full result as a physical table, equivalent to a materialized view.
4. Add `CURRENT_TIMESTAMP AS last_refreshed_at` to the SELECT inside the materialized view definition. Every refresh updates this timestamp to the refresh execution time. Alternatively, store the refresh timestamp in a separate `mv_refresh_log` table and expose it via a separate query.

🔗 **CHECKPOINT QUESTION**

You are designing the analytics layer for an e-commerce data warehouse in Snowflake. You have these reporting needs: (1) daily revenue by region — 1-hour staleness acceptable; (2) real-time order status for customer service; (3) analyst self-service queries on customer data — must not expose PII; (4) monthly product category performance — 24-hour staleness acceptable, very expensive query (120 seconds). For each, specify: regular view, Snowflake materialized view, or dbt model — and justify your choice.

PRACTICE PROBLEMS — SECURITY VIEW AND MATERIALIZED VIEW STRATEGY

ChatGPT • AI Assistant

A two-panel cost comparison diagram. Left panel (red background, label 'BEFORE — Regular View'): Shows 2,000 dashboard queries per day timeline. Each query arrow triggers the full 25-second GROUP BY execution on the 50M orders table. Cost calculation box: '2,000 queries × 25 seconds = 50,000 seconds of compute per day.' A database CPU spike icon at each query. Right panel (green background, label 'AFTER — Materialized View'): Shows the same 2,000 dashboard queries plus 24 hourly refresh arrows. Dashboard queries now point directly to a small 'mv_daily_revenue (90 rows)' box with <5ms label. Refresh arrows go to the large orders table once per hour. Cost calculation box: '24 refreshes × 30 seconds = 720 seconds + 2,000 queries × 0.005 seconds = 730 seconds total per day.' Savings badge: '98.5% compute reduction — 49,270 seconds saved per day.' Timeline strip at bottom shows the Airflow dependency chain: 'hourly_batch_load → (wait for completion) → refresh_mv_task.' Clean two-panel layout, red/green color scheme, timeline elements.

Practice Problem 2 — Materialized View Strategy Cost Comparison

sql — Practice Problems — Security View and Materialized View Strategy
Complete Solutions

```
-- =====
-- PROBLEM 1: Security view for analyst_ro role
-- =====

CREATE OR REPLACE VIEW customers_analyst AS
SELECT customer_id,
       name,
       city,
       tier,
       signup_date
FROM   customers
WHERE  is_active = TRUE
WITH CHECK OPTION;

-- Excludes: email, phone, national_id (PII)
-- Filters: active customers only
-- WITH CHECK OPTION: protects against invisible-row inserts

CREATE ROLE analyst_ro;
GRANT SELECT ON customers_analyst TO analyst_ro;
REVOKE SELECT ON customers          FROM analyst_ro;

-- Verify: connect as analyst_ro
-- SELECT * FROM customers_analyst LIMIT 3;      → works
-- SELECT email FROM customers LIMIT 1;          → ERROR: permission
-- denied

COMMENT ON VIEW customers_analyst
IS 'PII-safe view. Excludes email, phone, national_id. Active
only.';
```

```
-- Snowflake version
CREATE OR REPLACE VIEW customers_analyst AS
SELECT customer_id, name, city, tier, signup_date
FROM   customers WHERE is_active = TRUE;
GRANT SELECT ON VIEW customers_analyst TO ROLE analyst_ro;

-- =====
-- PROBLEM 2: Materialized view for daily revenue dashboard
-- =====

-- Step 1: Create MV with 90-day filter + staleness column
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS
SELECT
    o.region,
    DATE_TRUNC('day', o.order_date)      AS order_day,
    COUNT(DISTINCT o.order_id)           AS total_orders,
    COUNT(DISTINCT o.customer_id)       AS unique_customers,
    SUM(o.total_amount)                  AS total_revenue,
    AVG(o.total_amount)                  AS avg_order_value,
    CURRENT_TIMESTAMP                     AS last_refreshed_at
FROM   orders o
WHERE  o.status ≠ 'cancelled'
      AND o.order_date ≥ CURRENT_DATE - INTERVAL '90 days'
GROUP BY o.region, DATE_TRUNC('day', o.order_date);

-- Step 2: UNIQUE index (required for CONCURRENTLY)
CREATE UNIQUE INDEX idx_mv_rev_unique
    ON mv_daily_revenue_by_region(region, order_day);

-- Step 3: Verify row count (small result set)
SELECT COUNT(*) FROM mv_daily_revenue_by_region;

-- Step 4: Dashboard query (instant)
SELECT region, order_day, total_revenue, total_orders,
last_refreshed_at
FROM   mv_daily_revenue_by_region
WHERE  order_day ≥ CURRENT_DATE - 30
ORDER BY order_day DESC, total_revenue DESC;

-- Step 5: Airflow task (CONCURRENTLY – non-blocking)
-- task: PostgresOperator
-- sql: 'REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_revenue_by_region;'
-- depends_on_past: False
-- upstream: hourly_orders_batch_load_task

-- Cost comparison:
```

```
-- Before: 2000 × 25s = 50,000s/day
-- After: 24 × 30s + 2000 × 0.005s = 720 + 10 = 730s/day (98.5%
reduction)
```

Student

Can we go through the two practice problems?

ChatGPT • AI Assistant

CONCEPT CARD

These two problems are the most common real-world view tasks in data engineering interviews and on-the-job work.

Practice Problem 1 — Create a security view for a read-only analyst role:

Requirements:

- The **customers** base table has: *customerid, name, email, phone, nationalid, city, signupdate, tier, isactive, created_at*
- Analysts need: *customerid, name, city, tier, signupdate* — that is all
- Analysts must NOT be able to access email, phone, national_id (PII)
- Analysts should only see ACTIVE customers (*is_active* = TRUE)
- Any data written through the view must maintain the *is_active* = TRUE constraint
- The role is **analyst_ro** (read-only)

Solution:

sql — Code Example

```
-- Step 1: Create the security view
CREATE OR REPLACE VIEW customers_analyst AS
SELECT
    customer_id,
    name,
    city,
    tier,
    signup_date
FROM    customers
WHERE   is_active = TRUE
WITH CHECK OPTION; -- prevents inserts of inactive customers through
```

```

this view

-- Step 2: Create the read-only role (if not exists)
CREATE ROLE analyst_ro;

-- Step 3: Grant SELECT on the view to the role
GRANT SELECT ON customers_analyst TO analyst_ro;

-- Step 4: Explicitly revoke access to the base table
-- (if the role has schema-level SELECT, revoke it)
REVOKE SELECT ON customers FROM analyst_ro;

-- Step 5: Verify the analyst sees only safe columns
-- Connect as analyst_ro and test:
-- SELECT * FROM customers_analyst LIMIT 5;
-- → Shows: customer_id, name, city, tier, signup_date
-- SELECT * FROM customers LIMIT 5;
-- → ERROR: permission denied for table customers

-- Step 6 (optional): Add comment for documentation
COMMENT ON VIEW customers_analyst
    IS 'PII-safe customer view for analyst_ro role.
        Excludes: email, phone, national_id.
        Filters: is_active = TRUE only.
        Last updated: 2026-06-01';

-- For Snowflake:
CREATE OR REPLACE VIEW customers_analyst AS
SELECT customer_id, name, city, tier, signup_date
FROM   customers
WHERE  is_active = TRUE;

-- Grant in Snowflake
GRANT USAGE ON DATABASE analytics_db TO ROLE analyst_ro;
GRANT USAGE ON SCHEMA public TO ROLE analyst_ro;
GRANT SELECT ON VIEW customers_analyst TO ROLE analyst_ro;
-- Do NOT grant on the base customers table

```

Practice Problem 2 — Design a materialized view strategy for a daily revenue by region dashboard:

Context:

- The dashboard shows: daily revenue by region for the last 90 days
- Source table: **orders** — 50 million rows, growing by ~100,000 rows per day via hourly batch loads (not continuous streaming)

- The underlying GROUP BY query takes 25 seconds on the full table
- The dashboard is queried ~2,000 times per day by 150 BI users
- Business requirement: data should be no more than 1 hour stale
- Platform: PostgreSQL (not Snowflake for this problem)

Solution — Full strategy:

sql – Code Example

```
-- Step 1: Create the materialized view
CREATE MATERIALIZED VIEW mv_daily_revenue_by_region AS
SELECT
    o.region,
    DATE_TRUNC('day', o.order_date)                AS order_day,
    COUNT(DISTINCT o.order_id)                     AS total_orders,
    COUNT(DISTINCT o.customer_id)                  AS unique_customers,
    SUM(o.total_amount)                            AS total_revenue,
    AVG(o.total_amount)                            AS avg_order_value,
    MIN(o.total_amount)                            AS min_order_value,
    MAX(o.total_amount)                            AS max_order_value,
    CURRENT_TIMESTAMP                              AS last_refreshed_at
FROM    orders o
WHERE   o.status ≠ 'cancelled'
        AND o.order_date ≥ CURRENT_DATE - INTERVAL '90 days'
GROUP BY o.region, DATE_TRUNC('day', o.order_date)
ORDER BY order_day DESC, total_revenue DESC;

-- Step 2: Create UNIQUE index for CONCURRENTLY refresh
CREATE UNIQUE INDEX idx_mv_rev_region_day
    ON mv_daily_revenue_by_region(region, order_day);

-- Step 3: Create additional index for fast filtered reads
CREATE INDEX idx_mv_rev_region
    ON mv_daily_revenue_by_region(region);

-- Step 4: Verify initial population
SELECT COUNT(*) FROM mv_daily_revenue_by_region;
-- Expect: ~90 days × N regions rows (small result set)

-- Step 5: Refresh strategy decision
-- Requirement: < 1 hour stale
-- Batch load: hourly (new data arrives once per hour)
-- Conclusion: refresh 5 minutes AFTER each hourly batch load
--              completes
-- Example: if batch loads at :00 and completes by :10, refresh at :15
```

```
-- Step 6: Airflow DAG
-- Task 1: run_hourly_order_load (batch ingest job, runs at :00)
-- Task 2: refresh_mv (runs after Task 1 completes)

-- Step 7: The Airflow REFRESH task (non-blocking)
-- sql = 'REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_revenue_by_region;'

-- Step 8: Dashboard query (instant reads)
SELECT region,
       order_day,
       total_revenue,
       total_orders,
       last_refreshed_at
FROM   mv_daily_revenue_by_region
WHERE  order_day ≥ CURRENT_DATE - 30
ORDER BY order_day DESC, total_revenue DESC;
-- Response time: < 5ms (reading 30 × N regions stored rows)

-- Step 9: Cost comparison
-- BEFORE (regular view): 2000 queries × 25 seconds = 50,000
seconds/day
-- AFTER (materialized view): 24 refreshes × 30 seconds + 2000 × 0.005
seconds
--                               = 720 seconds + 10 seconds = 730
seconds/day
-- Savings: 49,270 seconds/day = 98.5% compute reduction
```

Justification for design decisions:

code – Code Example

CONCURRENTLY over blocking REFRESH:

Dashboard has 150 users querying throughout the day.
A 30-second blocking refresh would cause noticeable
'loading' pauses. CONCURRENTLY keeps reads available.

90-day WHERE filter in MV definition:

The dashboard only shows 90 days. Including older data
adds storage and refresh cost with no benefit.
Full historical data stays in the base orders table.

last_refreshed_at column:

Lets the dashboard display 'Data as of: 14:15'
so users understand the data age without guessing.

Refresh 5 minutes after batch load (not at fixed clock time):
If we refresh at :00 and the batch load isn't done until :10,
the refresh captures stale data. Scheduling refresh as a
downstream Airflow dependency of the batch load guarantees
the MV always reflects the latest completed load.

SUMMARY CARD

SQL 1.10 Key Takeaways:

- **Regular view** = virtual table, stored SELECT definition, no data on disk. Always fresh. Re-executes on every query.
- **Materialized view** = physical table, stored result rows on disk. Instant reads. Stale until refreshed.
- **View use cases:** encapsulation, security (PII masking), maintainability (fix logic in one place).
- **MV use cases:** expensive aggregations, dashboards with acceptable staleness, frequently queried summaries.
- **Updatable views:** single table, no GROUP BY/DISTINCT/aggregates/JOINS. Rare in data engineering.
- **WITH CHECK OPTION:** prevents inserts/updates that would make the row invisible in the view filter.
- **REFRESH MATERIALIZED VIEW:** blocking — locks during refresh. Use for low-traffic or overnight batches.
- **REFRESH MATERIALIZED VIEW CONCURRENTLY:** non-blocking — requires UNIQUE index. Use for production dashboards.
- **Snowflake MVs:** auto-refresh, no REFRESH command, credits charged, CLUSTER BY for micro-partition pruning, transparent query rewrite.
- **Decision rule:** < 5 seconds → regular view with index. > 5 seconds + staleness acceptable → materialized view.
- **dbt equivalents:** `materialized: view` = regular view. `materialized: table` = materialized view. `materialized: incremental` = incremental MV.

🌟 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior data engineer. I just learned SQL views and materialized views — regular views, updatable views with WITH CHECK OPTION, materialized views with blocking and CONCURRENTLY refresh, Snowflake auto-refresh MVs with CLUSTER BY, and the decision framework for when to use each. Give me 5 progressively

harder exercises. Each should present a real data engineering scenario with: table schema, row counts, query patterns, staleness requirements, and platform (PostgreSQL or Snowflake). Ask me to: decide which view type to use, write the CREATE statement, design the refresh strategy (frequency, method, Airflow DAG), and identify any indexing needed. After I answer: show the optimal solution, explain each decision, and give one follow-up question about edge cases (concurrent access, stale data communication, PII compliance). Do not continue until I submit my answer.'

🔗 CHECKPOINT QUESTION

Final review — you are joining a data team at an e-commerce company. They have a `customer_lifetime_value` view that joins customers, orders, order_items, and products across 50M orders. Every time the BI team queries it, it takes 90 seconds and spikes the database CPU. The data needs to be refreshed every 4 hours. The platform is Snowflake. Design the complete solution: view type, exact SQL, clustering strategy, refresh mechanism, cost safeguard, and how you communicate data freshness to the BI team.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.10 — views and materialized views, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Regular View vs Materialized View — Core Difference:** A two-panel comparison diagram. Left panel (blue background, label 'REGULAR VIEW...)